CoachPro — System Architecture (v1)

Oct 4, 2026 · prepared with @Ajith

Overview & scope

This is a multi-tenant SaaS web app that personal trainers use to **monitor their clients' health and fitness**. The trainer records and tracks everything for each client — body metrics, health markers, goals, and training — in one place, over time. Clients don't use the app; it is the trainer's tool for keeping a complete picture of each person they coach.

The core jobs the system does:

- Give each trainer a private workspace holding all their clients, isolated from every other trainer.
- Keep a full, dated record per client: weight and body measurements, health markers (blood sugar, cholesterol, vitamin D, and the like), and health and workout goals.
- Log the training the client is doing and show progress against goals over time.
- Store media and files — progress photos, and optionally a workout the trainer uploads to share with a client via a link.
- Scale smoothly from one trainer with a few clients to thousands of trainers each monitoring 50–60 clients, with no rebuild.

Only one user type sits at the centre: the **trainer** (the paying account holder). **Clients** are people the trainer manages inside the product — they do not log in or use the app themselves. The design still treats clients as first-class data, so a client-facing app could be added later without rework.

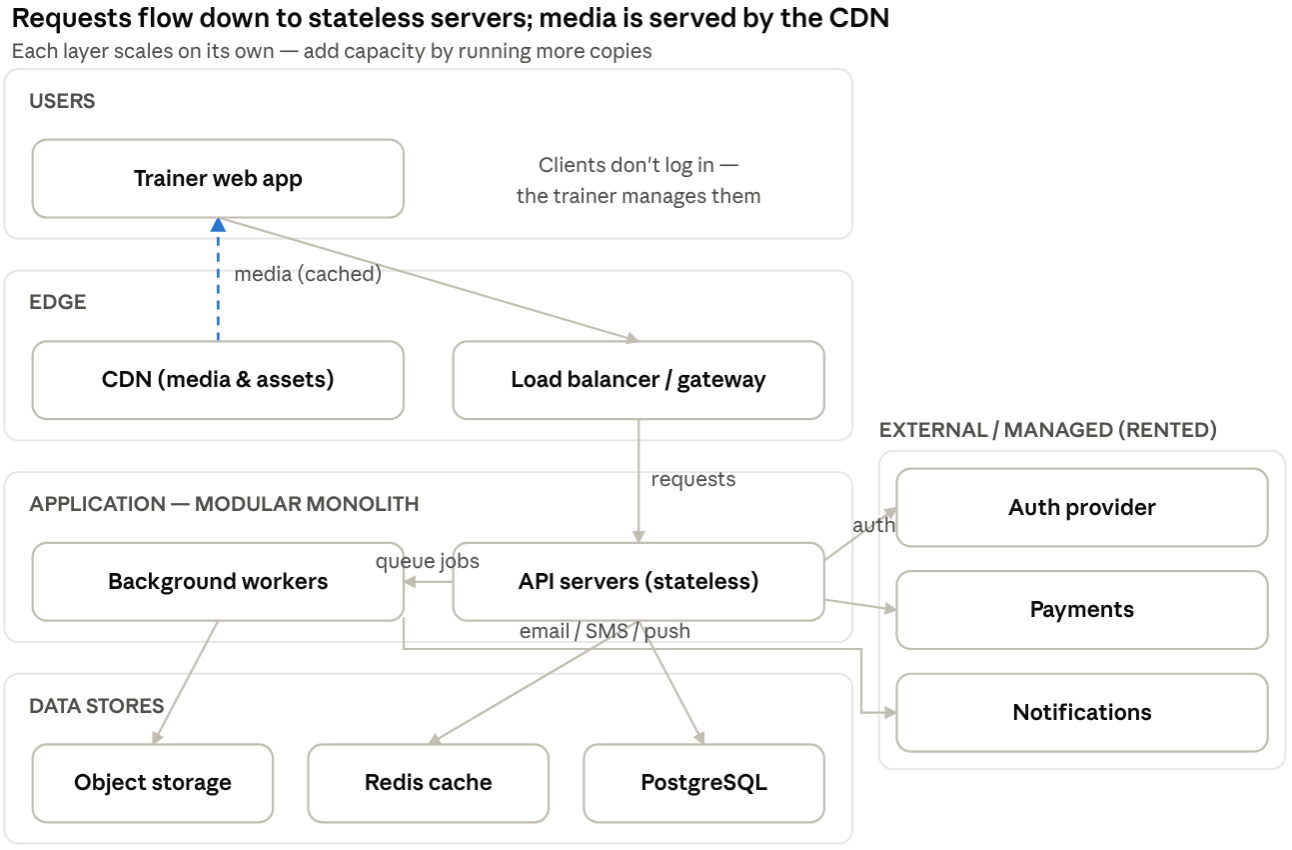
Assumptions

These assumptions shape every choice below. Correct any that are wrong and the design shifts with them.

- **Trainers are the only users.** The product is a monitoring tool — the trainer records and tracks all of each client's data; clients don't log in or use the app.
- The trainer enters everything: weight, measurements, health markers (blood sugar, cholesterol, and so on), goals, and the training done.
- Sharing a plan with a client is **optional and outside the core** — at most, the trainer uploads a file and shares a link; the system just needs somewhere to store it.
- Web-first, but built **API-first** — so a trainer mobile app, or a client-facing app later, can attach without rework.

- Scale to plan for: thousands of trainers, each monitoring 50–60 clients' worth of data — big enough to need a clean design, small enough not to need exotic infrastructure.
- A small team to start, so the design leans on managed cloud services; the data is sensitive health information, so privacy controls matter from day one.

The big picture



System architecture · 5 layers, request path + media path

Read the diagram top to bottom — that is the journey of a single request.

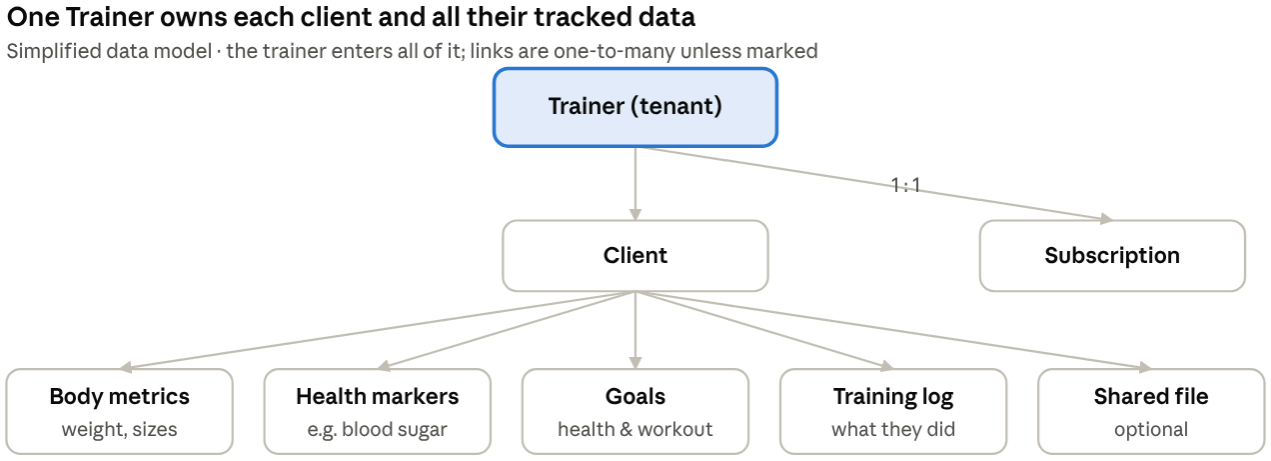
- **Users (top):** only the trainer uses the app — clients don't log in; the trainer manages them. The backend is API-first, so a trainer mobile app (or a client app later) attaches with no redesign.
- **Edge:** every request first hits a load balancer, which spreads traffic across your servers. Media is served separately by a CDN (a global cache), so it never touches your core servers.
- **Application:** your product logic lives in the API servers. They are stateless, so any server can handle any request and you scale just by running more of them. Slow work is handed to background workers through a queue, keeping the app responsive.

- **Data stores:** PostgreSQL is the system of record; Redis caches hot data for speed; object storage holds the large files.
- **External services:** things you rent rather than build — login, notifications, payments.

The one idea worth keeping: the request path (down the middle) and the media path (dashed, via the CDN) are deliberately separate, so your heaviest traffic — media (photos and files) — never slows the core system.

Data model & multi-tenancy

The single most important idea in the whole product is **multi-tenancy** — how you keep every trainer's clients and data completely separate from every other trainer's.



Simplified data model · everything hangs off the Trainer

The approach: one shared database where **every record carries a trainer_id**. The application always filters by the logged-in trainer, so Coach A can never see Coach B's clients — even though their data sits in the same tables. This is the standard, cost-effective pattern, and it scales to very large numbers of trainers.

The core entities, in plain terms:

- **Trainer** — the account holder and tenant; everything else belongs to one trainer.
- **Client** — a person the trainer monitors (a record the trainer maintains, not a user who logs in).
- **Body metrics** — weight and body measurements, recorded over time.
- **Health markers** — clinical readings the trainer tracks: blood sugar, cholesterol, vitamin D, blood pressure, and similar.
- **Goals** — health and workout targets, with progress against them.
- **Training log** — what the client is doing: workouts, sets, reps, effort.

- **Shared file** — optional: a workout the trainer uploads to share with a client via a link.
- **Subscription** — what the trainer pays you.

This is a **relational** shape — clear parent-child links everywhere — which is exactly what PostgreSQL is built for, and why it is the recommended database.

How it scales

Scalability here comes down to reaching for a small set of well-understood levers, in order, as load grows. You rarely need all of them early.

1. **Stateless app servers behind a load balancer.** The servers hold no session state, so you scale by running more copies and letting the load balancer spread traffic. This is the main lever and carries huge growth on its own.
2. **Cache hot data in Redis.** Things read constantly — a trainer's roster, a client's recent records, login tokens — are served from memory instead of hitting the database every time.
3. **Add read replicas to the database.** Most traffic is reads (viewing client records, history, progress). Copies of the database handle those reads while the main one handles writes.
4. **Push slow work to background workers.** Media processing, emails, reminders, and reports run off a queue, away from the user's request, so the app stays fast under load.
5. **Serve all media through a CDN.** Photos and files come from object storage via a global cache, never through your app servers — keeping the heaviest traffic off your core system entirely.
6. **Partition the database by trainer — much later, if ever.** Because every record already belongs to one trainer, the data can eventually be split across databases by trainer. Most products never need this; it is the ceiling, not the starting point.

The guiding principle: start as a single, well-organised backend (a "modular monolith"), not microservices. One clean codebase is faster to build and easier to run for a small team, and the levers above scale it a long way. Split a piece into its own service only when a real bottleneck — or team size — forces it.

Build vs buy, and a starter stack

For a small team, the fastest safe path is to **buy** (use managed services for) the hard, undifferentiated infrastructure, and **build** only what makes your product distinct — the per-client tracking and monitoring that trainers rely on. Here is one concrete, widely-used, easy-to-hire-for stack.

Layer	Recommendation	Why
Frontend	Next.js (React) + TypeScript	Large hiring pool; serves web now and shares code with a future mobile app
Backend API	Node.js (NestJS) or Python (Django / FastAPI)	Pick by your team's skills; both scale fine as a modular monolith
Database	Managed PostgreSQL (RDS / Aurora, Supabase, Neon)	Relational fits the trainer → client → metrics, goals and logs links; managed removes ops burden
Cache	Managed Redis	In-memory speed for hot reads, sessions, rate limiting
Media storage	S3 (or Cloudflare R2) + CDN	Cheap, durable storage for video and photos, served fast worldwide
Auth	Managed (Clerk, Auth0, Supabase Auth)	Secure login and roles for your trainers, without building it yourself
Background jobs	Managed queue + workers (e.g. SQS)	Runs slow work off the request path
Hosting	A PaaS to start (Render, Railway, Fly), move to AWS / GCP as you grow	Minimise ops early, keep room to grow
Monitoring	Sentry (errors) + the host's metrics and logs	See problems before customers do

None of these are locked in — they are a sensible default a new team can ship on quickly and swap out later.

Security, privacy & compliance

Because you hold sensitive personal health data — body metrics, clinical markers, photos, goals — privacy is a first-class design concern, not an afterthought.

The essentials:

- **Tenant isolation.** Enforce at the application layer (and ideally the database) that a trainer can only ever read or write their own clients' data. This is the single most important control in a multi-tenant product.

- **Sensitive health data.** Blood markers and similar clinical readings are among the most sensitive data you can hold — store only what the trainer needs, keep it encrypted, and control and log who can see it.
- **Encryption everywhere.** Data encrypted in transit (HTTPS) and at rest (database and file storage) — standard with managed services.
- **Access control.** Clear roles (trainer and admin) with least-privilege access; media and shared files served via short-lived signed URLs, never public links.
- **Consent and data rights.** You hold clients' personal data on their trainer's behalf, so you still need consent for collecting it and a way to export or delete it on request.

On regulation: you are based in India, so the **Digital Personal Data Protection (DPDP) Act** governs how you collect and handle personal data. If you take clients in the EU or UK, **GDPR** applies too (consent, data access, deletion, breach notification). Neither needs exotic infrastructure — mostly consent flows, clear policies, and the controls above. Worth a short legal review before launch rather than after.

Suggested build sequence

You don't build all of this at once. A sensible order ships value early while keeping the architecture intact:

1. **Phase 1 — Core tracker (MVP).** Trainer login, client roster, and the per-client record: body metrics, health markers, and goals, entered by the trainer with full history. Backend + PostgreSQL + auth + basic hosting. This is the heart of the product.
2. **Phase 2 — Progress & training.** Logging the training done, progress charts against goals, and progress photos — bringing in media storage and the CDN.
3. **Phase 3 — Sharing (optional).** Let the trainer upload a workout file and share it with a client via a link. A small feature: just storage plus a signed link.
4. **Phase 4 — Payments & reminders.** Billing for trainer subscriptions, and reminders for the trainer (for example, to update a client) — bringing in the queue and workers.
5. **Phase 5 — Scale & polish.** Add caching, read replicas, search, and a trainer mobile app as usage justifies each.

Everything in Phases 2–5 attaches to the Phase 1 foundation without rebuilding it — which is the whole point of getting the architecture right first.

Decisions that change this design

A few answers will sharpen this from a strong default into the right design for your product. These genuinely move the architecture:

1. **Client access — decided.** Trainer-only; clients don't log in. The design stays API-first so a client app could be added later without rework.
2. **The tracked data — the key open item.** What exactly does the trainer record per client? A confirmed list of fields (metrics, markers, goals, and their units) defines the core data model and the main screens.
3. **Structured programs vs. files.** Do trainers need a built-in program and exercise builder, or is uploading a workout file and sharing a link enough (as you suggested)?
4. **Charts & views.** Which per-client views matter most — a weight trend, a marker over time, goal progress, a single dashboard?
5. **Reminders.** Do trainers want reminders (for example, to update a client or follow up), or is that out of scope for now?
6. **Payments & scale.** Trainer subscriptions only? And how many trainers in year one, with what team and budget? This tunes build-vs-buy.
7. **Geography.** India only, or trainers (and their clients' data) in the EU / UK / US too? This decides which privacy laws apply.

The one thing still to define is the exact list of fields the trainer records per client — the metrics, markers, goals, and their units. Send that and I'll turn it into a concrete, field-level data model and the core screens around it.